

Design and Implementation of a Context-Aware Student Enquiry Chatbot with Analytics-Driven Decision Support System

Case study: DELSU Student Enquiry System

Project type	B.Sc. Computer Science final-year project (MVP)
Frontend	Next.js (App Router), React, Material UI
Backend	Next.js API routes (Node.js runtime)
Database	Supabase (PostgreSQL) with the pgvector extension
AI provider	OpenAI (chat completions + text embeddings)
Deployment	Vercel

Table of Contents

1. Introduction
2. System Overview & User Roles
3. System Architecture
4. Database Design (ERD & Data Dictionary)
5. UML Design
 - 5.1 Use-Case Diagram & Descriptions
 - 5.2 Activity Diagram
 - 5.3 Sequence Diagram
 - 5.4 Class Diagram
6. Context-Aware Chatbot & RAG Pipeline
7. Frontend Design
8. Backend / API Reference
9. Authentication, Roles & Security
10. Administrator Analytics Dashboard
11. Functional Testing
12. Performance Evaluation
13. Deployment
14. Conclusion & Future Work

1. Introduction

1.1 Project Purpose

The system is a web-based student enquiry platform that helps prospective and current students obtain academic, admission, and institutional information through a conversational chatbot. It combines four capabilities: (i) a student enquiry chatbot, (ii) conversational context awareness, (iii) Retrieval-Augmented Generation (RAG) grounded in an institutional knowledge base, and (iv) analytics-driven decision support for administrators.

1.2 Scope

The implementation is a realistic MVP suitable for a final-year project: it is functionally complete for its stated goals, uses a small and well-justified set of technologies, and avoids speculative features. Where the original specification's suggested design (e.g. a document/chunk knowledge base) was not the simplest solution preserving the requirement, a simpler equivalent was used instead and is documented as such (see Section 5.4).

1.3 Objectives

- Allow students to submit enquiries through a chatbot interface and receive answers grounded in approved institutional information rather than the language model's general knowledge.
- Maintain conversational context so follow-up questions (e.g. “what about the deadline?”) are understood in relation to the preceding exchange.
- Support both anonymous (“prospective student”) and authenticated (“current student”) use, with the latter gaining a saved conversation History and a Profile.
- Give administrators a dashboard that turns stored enquiry data into decision-support information: categories, volumes, trends, and reply feedback.
- Enforce role-based access so students cannot reach administrator functionality.

2. System Overview & User Roles

Role	Capabilities	Restrictions
Prospective Student	Access the chatbot at /chat with no account; submit enquiries; ask a follow-up in the same browser session.	No saved History/Profile; cannot leave reply feedback (tied to an account); no admin access.
Current Student	Log in; chat; conversation History (/history); Profile (/profile) with academic details; leave thumbs up/down feedback with an optional comment.	Cannot reach any /admin/** route, even by typing the URL directly.
Administrator	Analytics dashboard; browse any student's enquiries; manage the knowledge base; create/delete student & admin accounts; edit app-wide settings.	Distinct login is not separate — the same /login page routes admins here by their account's role.

Core system flow

Student submits enquiry → system checks relevant previous conversation context → the enquiry is embedded → a pgvector cosine-similarity search runs against the knowledge base → relevant chunks are retrieved → the retrieved knowledge, current enquiry, and recent context are supplied to the language model → the model generates a response → the response is shown and the interaction stored → stored data feeds the analytics dashboard.

3. System Architecture

The system follows a layered, component-oriented architecture. Presentation, application logic, retrieval/analytics, and storage are kept as separate concerns even though they are co-located in one Next.js codebase (API routes serve as the application layer, avoiding a duplicate backend service for what is already a server-capable framework).

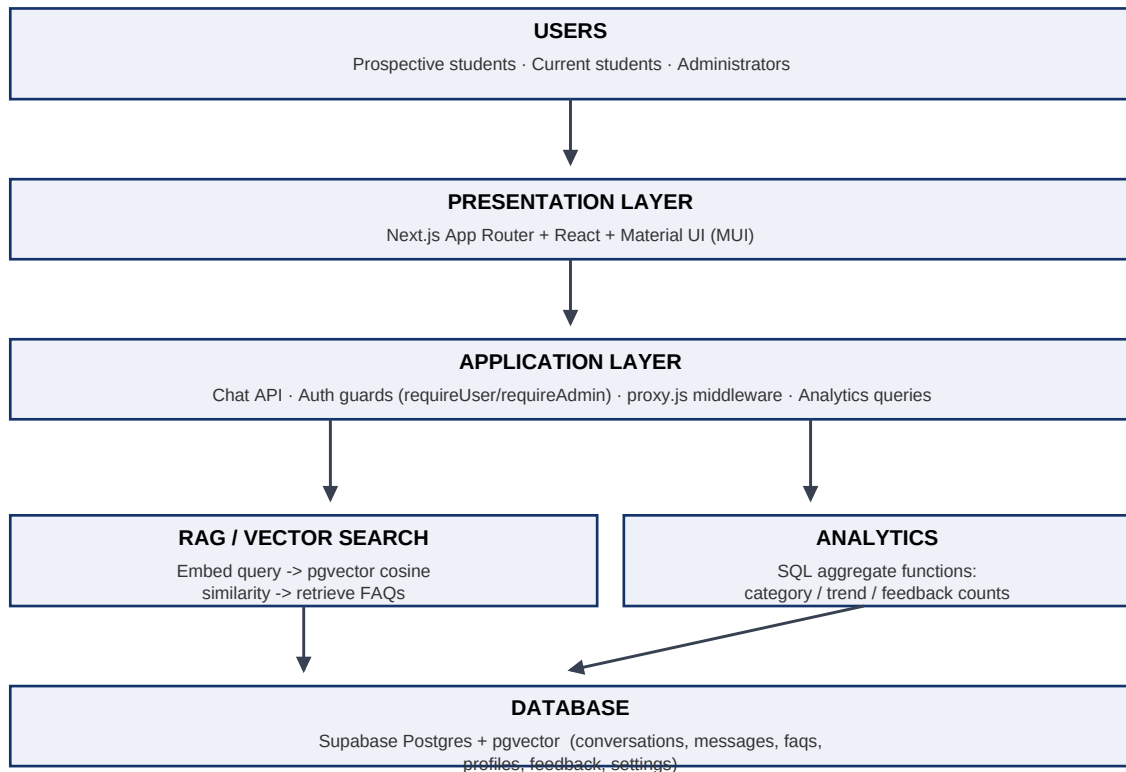


Figure 3.1 — System architecture

4. Database Design

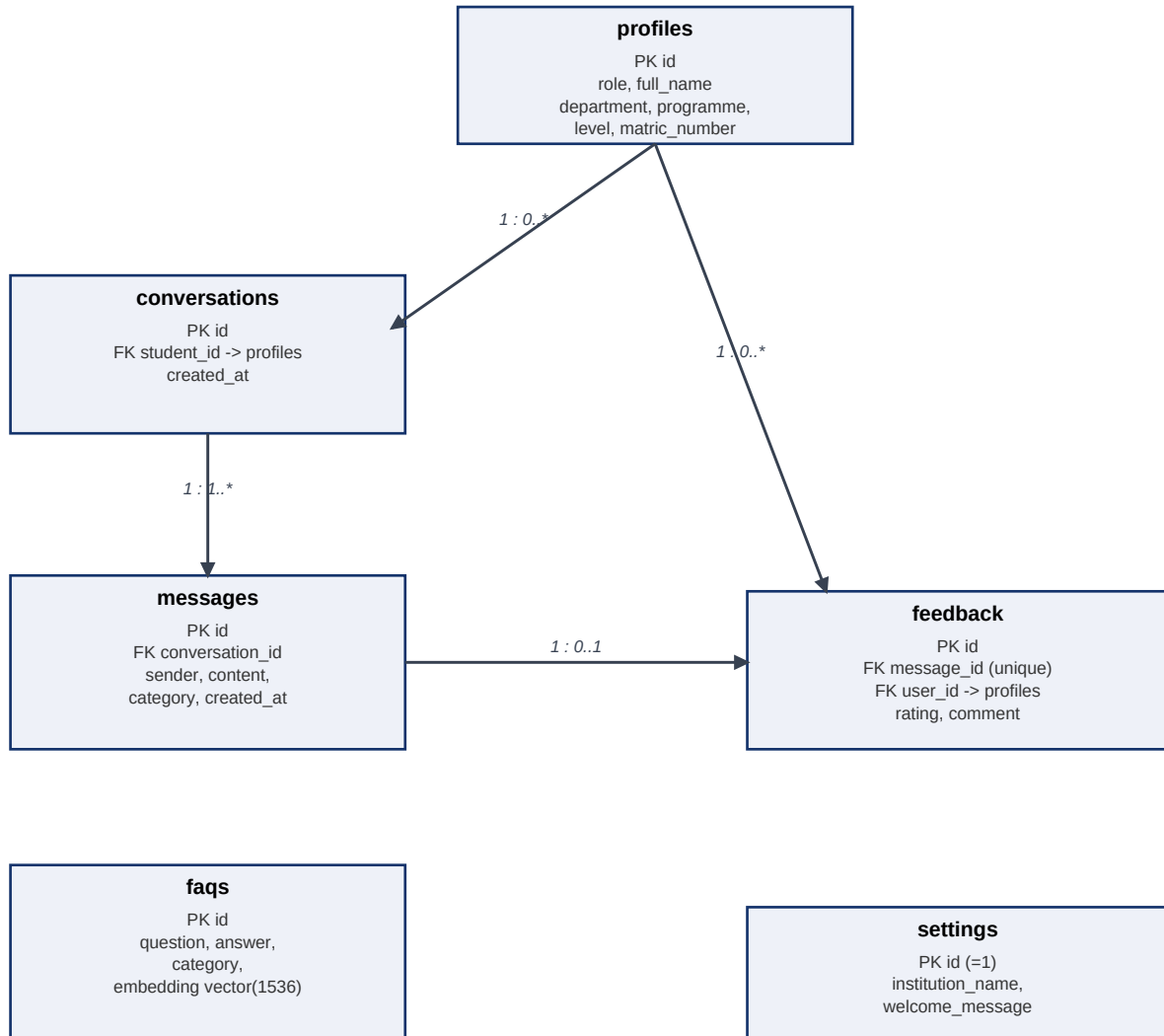


Figure 4.1 — Entity-relationship diagram

Data dictionary

profiles

Column	Type	Notes
id	uuid, PK	Same value as the Supabase Auth user id.
role	text	'student' or 'admin'. Mirrors the JWT's user_metadata.role, which is the actual access-control source of truth.
full_name, department, programme, level, matric_number	text	Academic profile fields; set by an admin when the account is created.

conversations

Column	Type	Notes
id	uuid, PK	
student_id	uuid, FK → profiles	Nullable — null for an anonymous/prospective-student conversation.

messages

Column	Type	Notes
id	uuid, PK	
conversation_id	uuid, FK	
sender	text	'student' or 'bot'.
category	text	Keyword-classified enquiry topic, set on student messages only; powers the analytics charts.

faqs (knowledge base)

Column	Type	Notes
id	uuid, PK	
question, answer, category	text	Approved institutional Q&A content.
embedding	vector(1536)	OpenAI text-embedding-3-small output; indexed with an ivfflat cosine index.

feedback

Column	Type	Notes
id	uuid, PK	
message_id	uuid, FK, unique	One rating per bot message.
user_id	uuid, FK → profiles	The student who left the feedback.
rating, comment	text	'up'/'down' plus an optional free-text comment.

settings

Column	Type	Notes
id	int, PK (=1)	A CHECK constraint enforces exactly one row.
institution_name, welcome_message	text	Admin-editable; used live by the chat page and the AI system prompt.

5. UML Design

Internal technical operations (embedding generation, cosine-similarity computation, the vector search itself) are deliberately not modelled as use cases below — they are implementation detail behind a single user-facing goal, “Submit Enquiry”.

5.1 Use-Case Diagram

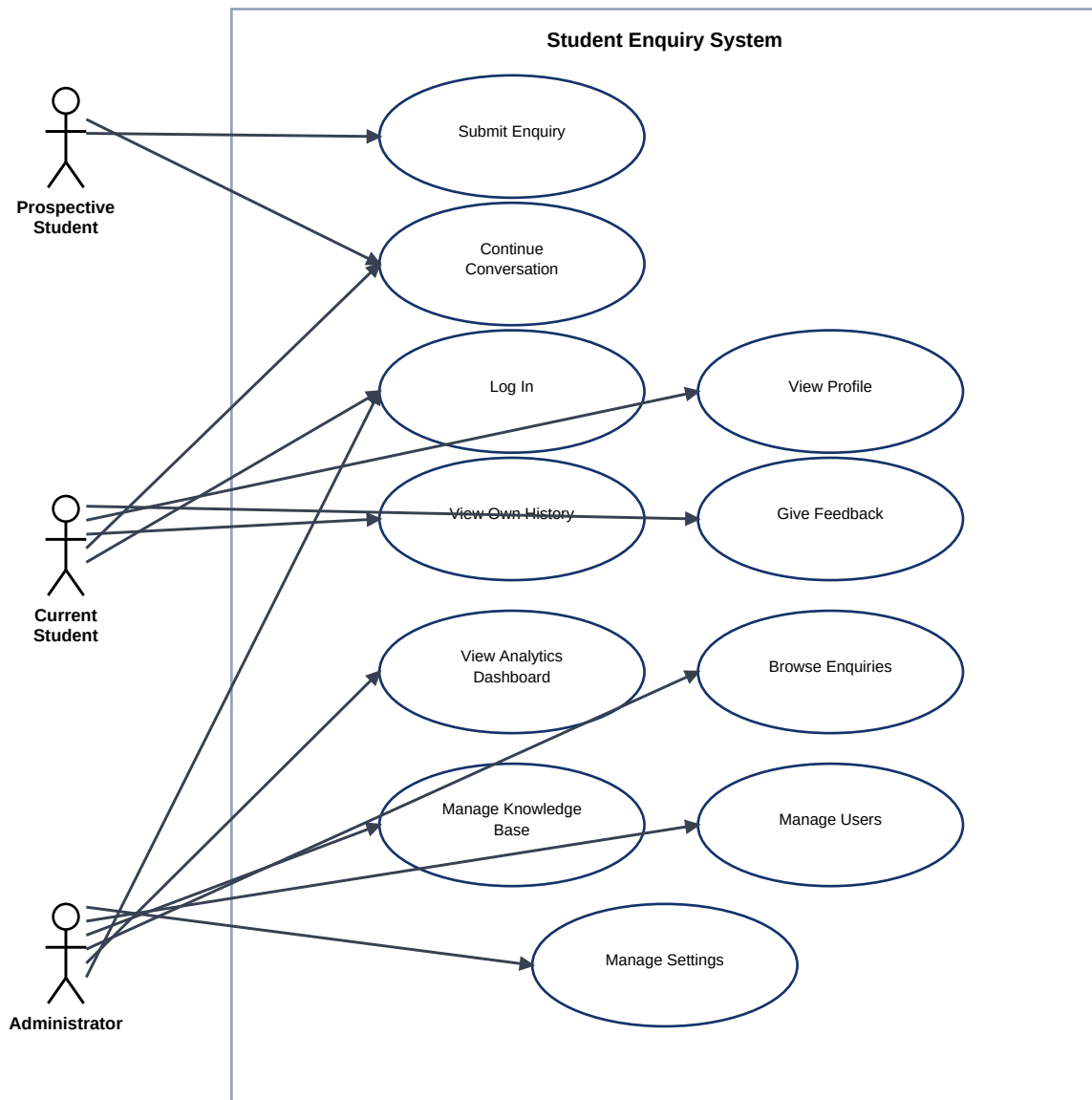


Figure 5.1 — Use-case diagram

Use case	Actor(s)	Description
Submit Enquiry	Prospective, Current Student	Type a question and send it; a stored, RAG-grounded reply is returned.
Continue Conversation	Prospective, Current Student	Ask a follow-up; the last 6 messages are used as context.
Log In	Current Student, Admin	Authenticate; routed to /chat or /admin/dashboard by role.
View Own History	Current Student	Browse and re-open past conversations.

Use case	Actor(s)	Description
View Profile	Current Student	See name, email, role, academic details.
Give Feedback	Current Student	Rate a bot reply, optionally with a comment.
View Analytics Dashboard	Administrator	KPI cards + category/trend charts, computed live.
Browse Enquiries	Administrator	Read-only view of any conversation.
Manage Knowledge Base	Administrator	Add/edit/delete FAQs; each is embedded automatically.
Manage Users	Administrator	Create/delete student & admin accounts.
Manage Settings	Administrator	Edit institution name & chatbot welcome message.

5.2 Activity Diagram

Swimlanes: Student, Chatbot System, RAG/Knowledge Base, Database.

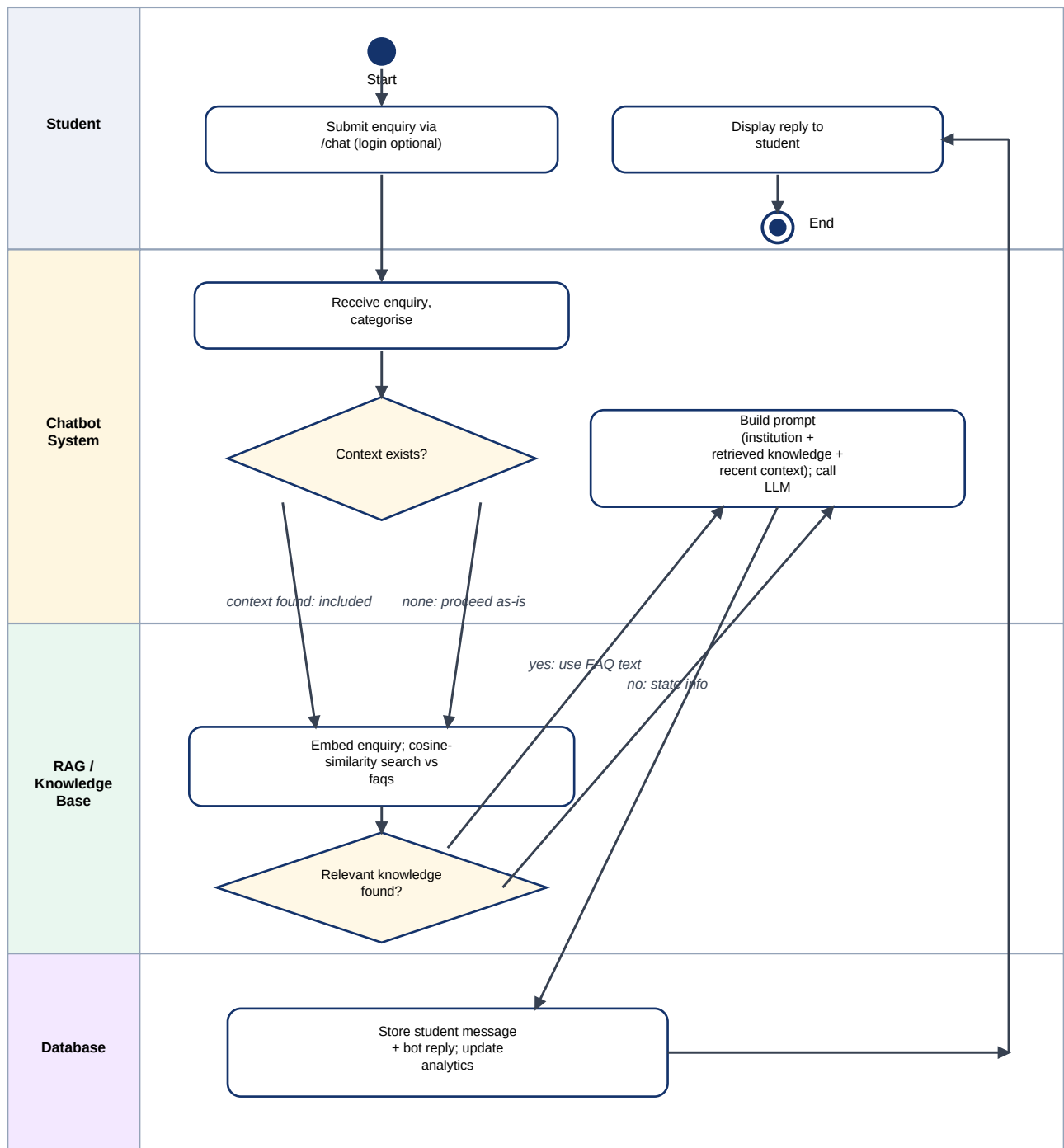


Figure 5.2 — Activity diagram of the enquiry-handling process

5.3 Sequence Diagram

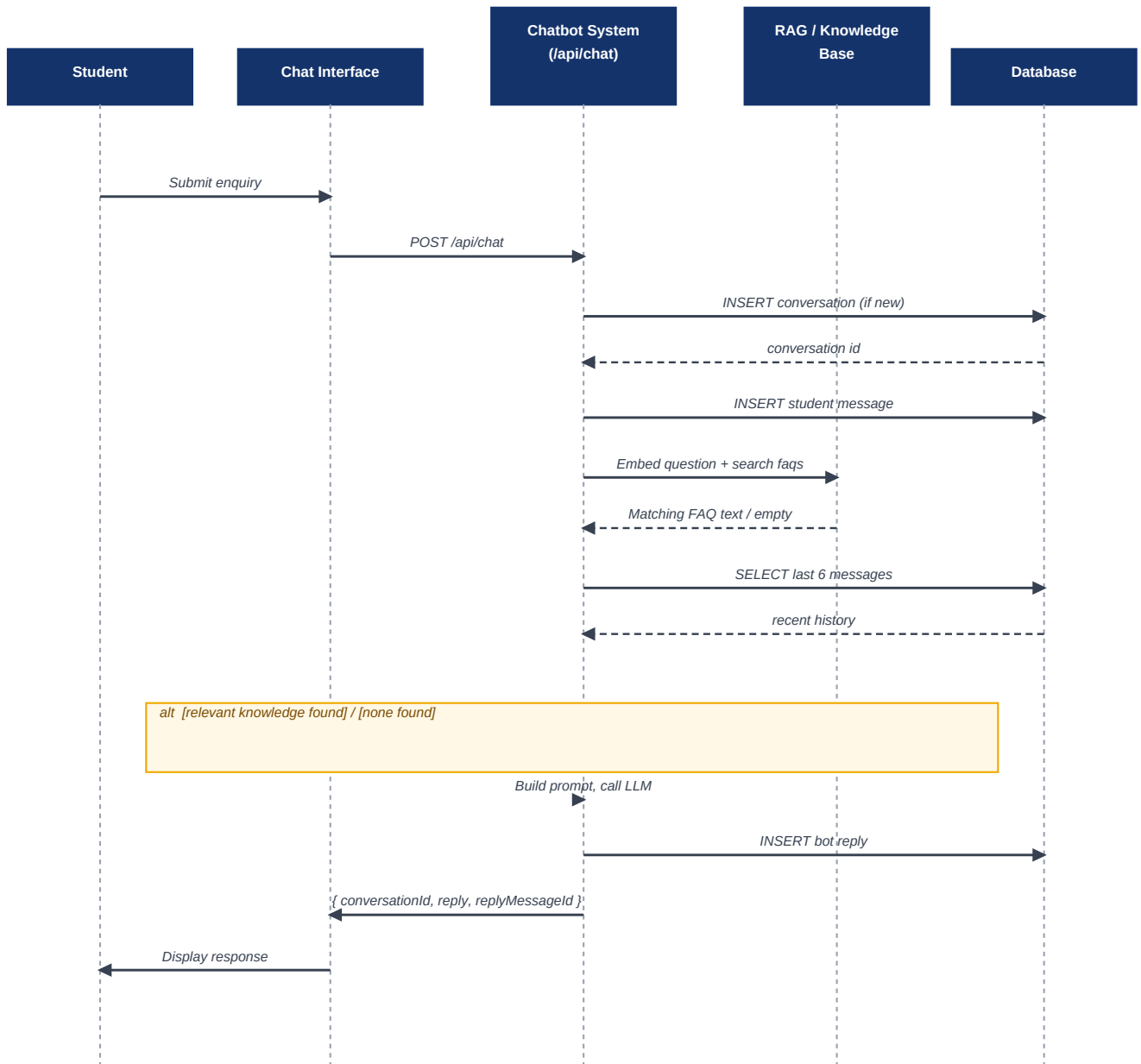


Figure 5.3 — Sequence diagram for one chat exchange

5.4 Class Diagram

Note on a deliberate deviation: a document/chunk knowledge-base model (separate KnowledgeDocument and DocumentChunk classes) was considered, but the implemented knowledge base is FAQ-entry based — each entry is short enough to embed directly, with no chunking step. Modelling a DocumentChunk class that does not exist in the code would misrepresent the implementation, so KnowledgeEntry below maps onto the real faqs table instead.

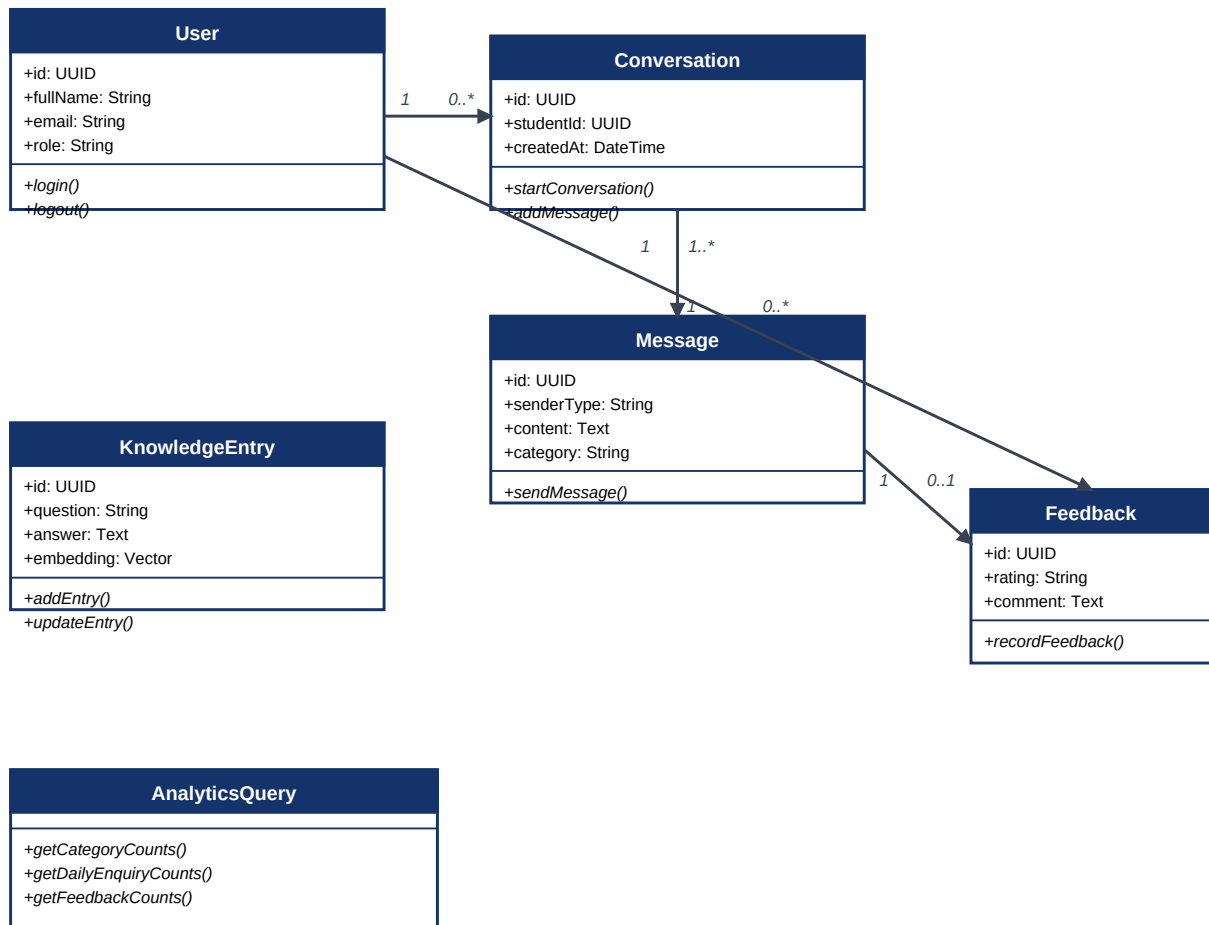


Figure 5.4 — Class diagram

6. Context-Aware Chatbot & RAG Pipeline

6.1 Context awareness

Every request identifies its conversation (creating one on the first message) and retrieves the most recent messages within it before generating a reply. Those messages are translated into the language model's role format and prepended to the prompt, so a follow-up such as “what about the deadline?” is interpreted using the immediately preceding exchange rather than as an isolated question. This is bounded context (the last 6 messages), not unlimited memory of the entire conversation history.

6.2 Retrieval-Augmented Generation

The knowledge base (approved institutional Q&A content) is embedded once, at write time, using OpenAI's text-embedding-3-small model (1536-dimensional vectors). Every incoming enquiry is embedded the same way, and pgvector's cosine-distance operator ranks knowledge-base entries by similarity:

$$\cos(\theta) = (A \cdot B) / (||A|| \cdot ||B||)$$

where A is the query embedding and B is a stored knowledge embedding. Entries above a similarity threshold are supplied to the language model as context; the system prompt explicitly instructs the model not to answer from outside that context. No TF-IDF or keyword-matching step is part of this pipeline — retrieval is embeddings → vector search → cosine similarity → ranked chunks → language model, exactly as specified.

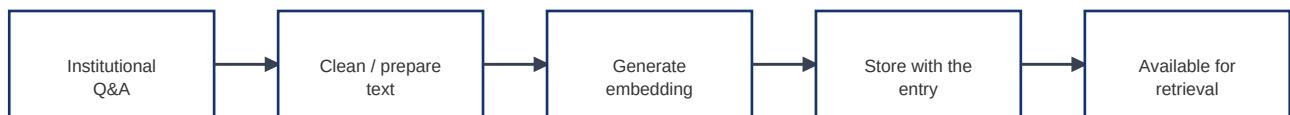


Figure 6.1 — Knowledge ingestion pipeline

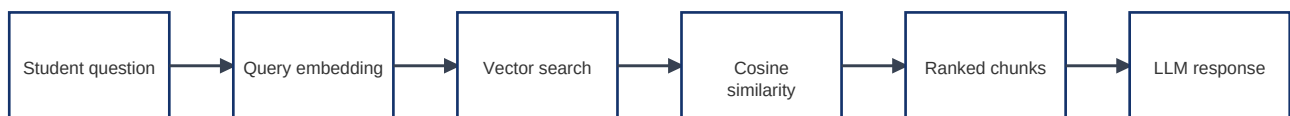


Figure 6.2 — RAG retrieval pipeline

7. Frontend Design

Built with Next.js (App Router) and Material UI, chosen over a plain client-side SPA because the project needs authenticated, role-gated areas and server-side data fetching (see Section 9) that a framework-less frontend would otherwise have to hand-roll.

Route	Purpose
/	Public landing page.
/login	Single login for students and admins; redirects by role after sign-in.
/chat	Chat widget; works signed-out (anonymous) or signed-in.
/history, /history/[id]	A signed-in student's own past conversations and their transcripts.
/profile	Signed-in student's account details.
/admin/dashboard	KPI cards + charts.
/admin/enquiries, /admin/enquiries/[id]	Browse any conversation.
/admin/knowledge-base	FAQ CRUD with automatic embedding.
/admin/users	Create/list/delete accounts.
/admin/settings	Institution name & welcome message.

UI design language: a deep navy primary colour, a warm gold accent, a light grey/off-white background, white cards with subtle borders, and Material UI's default sans-serif type scale. Both the student and admin sidebars collapse to a mobile top-bar-and-drawer pattern below the small-screen breakpoint.

8. Backend / API Reference

The “backend” is the set of Next.js API routes and server components under `app/api` and `app/**/page.js` — kept modular by responsibility (chat, feedback, faqs, admin/users, admin/settings) rather than as one monolithic handler.

Endpoint	Auth	Purpose
POST /api/chat	Optional	Submit an enquiry; returns the reply and its message id.
POST /api/feedback	Required (any account)	Upsert a rating (+ optional comment) for one bot message.
GET/POST/PUT/DELETE /api/faqs	Admin	Knowledge-base CRUD; embeds on save.
GET/POST/DELETE /api/admin/users	Admin	Account management; returns a one-time temp password on creation.
GET/PUT /api/admin/settings	Admin	Read/update the settings row.

Each chat request performs, in order: identify user (optional) → identify/create conversation → retrieve context → create query embedding → search the vector database → retrieve relevant chunks → build the prompt → generate the response → store both messages → record analytics (implicitly, via the stored category) → return the response — each stage its own function rather than one large handler.

9. Authentication, Roles & Security

Measure	Implementation
Authentication	Supabase Auth (email + password); no public sign-up — accounts are created by an admin only.
Role-based access	Role travels in the account's JWT (user_metadata.role); read with no extra database call.
Route protection	proxy.js (edge middleware) redirects logged-out visitors away from /history & /profile, and any non-admin away from /admin/**, before the page even renders.
API-level protection	Each admin API route independently re-checks role server-side (requireAdmin()), since middleware alone doesn't stop a direct request to the route.
Secrets	OPENAI_API_KEY and the Supabase secret key are read from environment variables server-side only; never referenced in any "use client" file, never present in the built client bundle (verified by direct inspection of build output), never committed to git.
Row Level Security	Enabled on every table with no permissive policies ("deny all"), as a safety net — all real access goes through the server-only secret-key client, which bypasses RLS by design.
Input validation	Empty/whitespace-only messages are rejected client-side (disabled send button) and server-side (400 response).

10. Administrator Analytics Dashboard

The dashboard is decision support, not decoration: every number is a live query against the same tables the chatbot writes to, computed by dedicated SQL functions rather than hard-coded.

KPI / chart	Source
Total Enquiries	get_category_counts() totals
Active Users	get_active_student_count(30) — distinct students who've chatted in the window
Knowledge Documents	count(*) on faqs
Feedback	get_feedback_counts() (up/down totals)
Top Enquiry Topics (bar chart)	get_category_counts()
Enquiry Trends (line chart)	get_daily_enquiry_counts(14)
Enquiries list	get_recent_conversations() — joins profiles for the student's name

11. Functional Testing

Results below reflect what was actually verified during development, or are explicitly marked pending where a live database with real accounts is required — no result is invented.

#	Test case	Expected result	Status
1	Greeting	Friendly acknowledgement, no fabricated facts	Pending manual QA
2	Admission enquiry	Reply grounded in a matching FAQ	Pending manual QA
3	Academic enquiry	Reply grounded in a matching FAQ	Pending manual QA
4	Follow-up enquiry (context)	Follow-up interpreted using prior context	Code path verified
5	No relevant knowledge	States it lacks the information; does not invent an answer	Code path verified
6	Out-of-scope enquiry	Redirected to the system's actual purpose	Pending manual QA
7	Prospective (anonymous) chat	Works with no login; no History entry created	Verified
8	Student authentication	Redirected to /chat; History/Profile reachable	Pending manual QA
9	Administrator authentication	Redirected to /admin/dashboard	Pending manual QA
10	Analytics dashboard display	Live figures, not placeholders	Code path verified
11	Access restriction (student to admin)	Redirected away from /admin/**	Verified
12	Access restriction (logged-out)	Redirected to /login	Verified
13	Invalid input (empty message)	Send blocked client- and server-side	Verified
14	Knowledge-base management	New FAQ usable by the chatbot with no code change	Pending manual QA
15	Feedback with comment	Row written; dashboard Feedback count updates	Pending manual QA
16	Error handling (misconfiguration)	Clean error message, not a stack trace	Verified

12. Performance Evaluation

The specification calls for accuracy, precision, recall, F1-score, and response time. **None of these have been measured yet** — computing them honestly requires a labelled evaluation set (realistic student questions with a human-judged expected answer), running each through the deployed chatbot once the knowledge base holds real institutional content, and a human grading each response. Reporting numbers without that process would mean inventing results, which this report deliberately does not do.

Metric	Formula	Result
Accuracy	Correct responses / Total queries	Not yet measured
Precision	$TP / (TP + FP)$	Not yet measured
Recall	$TP / (TP + FN)$	Not yet measured
F1-score	$2 \cdot P \cdot R / (P + R)$	Not yet measured
Avg. response time	Request-to-reply wall clock	Not yet measured
Context-retention rate	% of follow-ups answered correctly using prior context	Not yet measured

13. Deployment

The application is deployed to Vercel as a single unit (frontend, API routes, and edge middleware together); the database is a managed Supabase Postgres project. Environment variables (the OpenAI key and the Supabase secret key) are stored encrypted in the hosting platform and read only at request time on the server — never present in any file shipped to the browser.

Environment variables

Variable	Where used
NEXT_PUBLIC_SUPABASE_URL	Browser + server — project URL, not sensitive.
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Browser + server — public key, restricted by RLS.
SUPABASE_SECRET_KEY	Server only — bypasses RLS; the one client used for all application data.
OPENAI_API_KEY	Server only — embeddings + chat completions.

14. Conclusion & Future Work

The implemented system meets its stated objectives: a context-aware, RAG-grounded chatbot available to both anonymous and authenticated students, and an administrator back office that turns the same stored data into decision-support analytics, all behind role-based access control. The design deliberately favoured the simplest solution that preserves each requirement — a single Next.js codebase instead of a separate frontend/backend split, FAQ entries instead of a document-chunking pipeline the knowledge base doesn't need, and a manually shared temporary password instead of an email-invite system that would require additional infrastructure.

Future work

- Run the functional test suite (Section 11) against a populated knowledge base and record the performance metrics in Section 12 honestly, once real data exists.
- Replace the temporary-password account creation flow with Supabase's `inviteUserByEmail`, once an email provider is configured.
- Add an admin Reports page (data export) if a future requirement calls for it — the original specification marks this optional.
- Consider chunking for any future knowledge-base entries long enough to need it, rather than retrofitting a document/chunk model the current content does not require.